

Who Can Turn Off Your Ethereum App?

A Full-Stack Walk-Away Test

James Carnley

Ethereum File System



Building Cypherpunk Apps

Why make an app you can walk away from?

What happens when an app disappears?

- Users lose their work and trust
- Other developers can’t rely on that functionality
- Communities splinter

Software worth building on

For people

Keep years of useful work

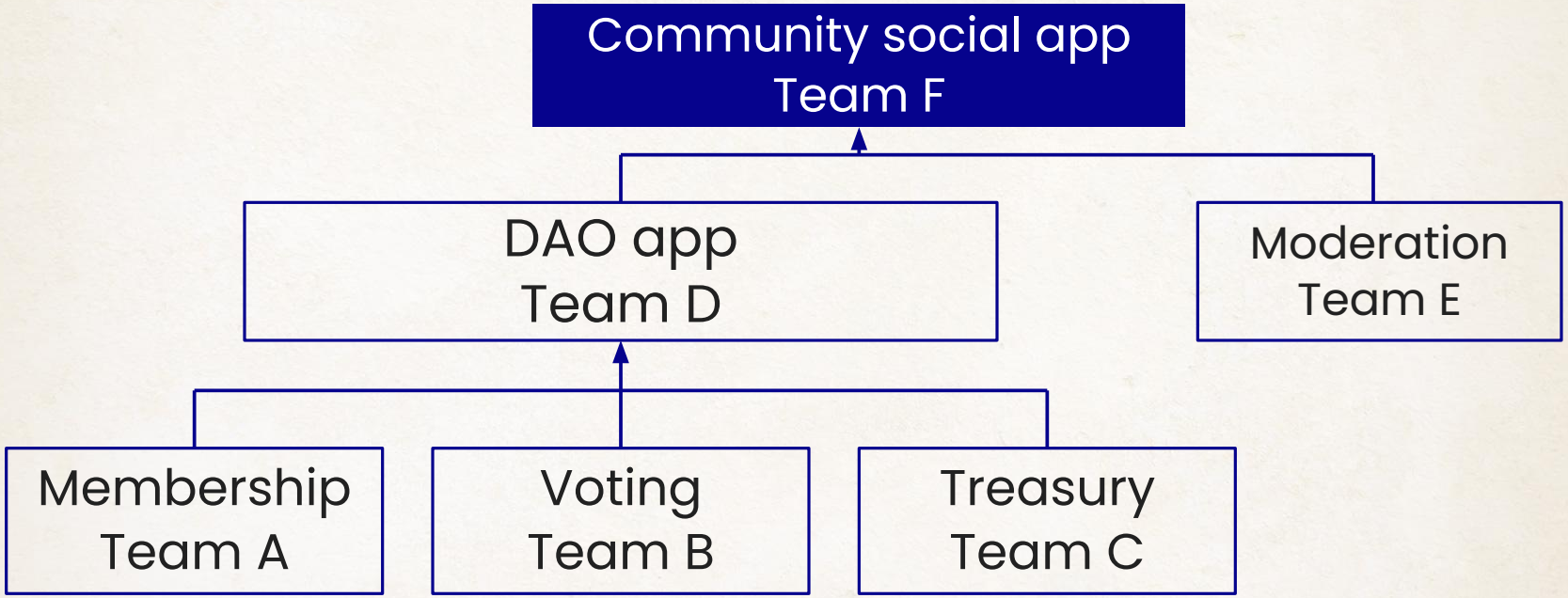
Choose better tools

For builders

Reuse dependable capabilities

Build bigger things together

Hyperstructures: build on each other



Illustrative architecture: each team adds a dependable capability

A walk-away test

The team disappears

Can users keep working?

Can others run the services?

Control becomes hostile

Can users keep their work?

Can they keep the rules they chose?

The application stack

Interface

- Names and discovery
- Runnable software
- Accounts and recovery

Data

- Storage and formats
- State and indexes
- Background work

Rules

- Contracts and governance
- Network and settlement



Interface

Finding and using the app

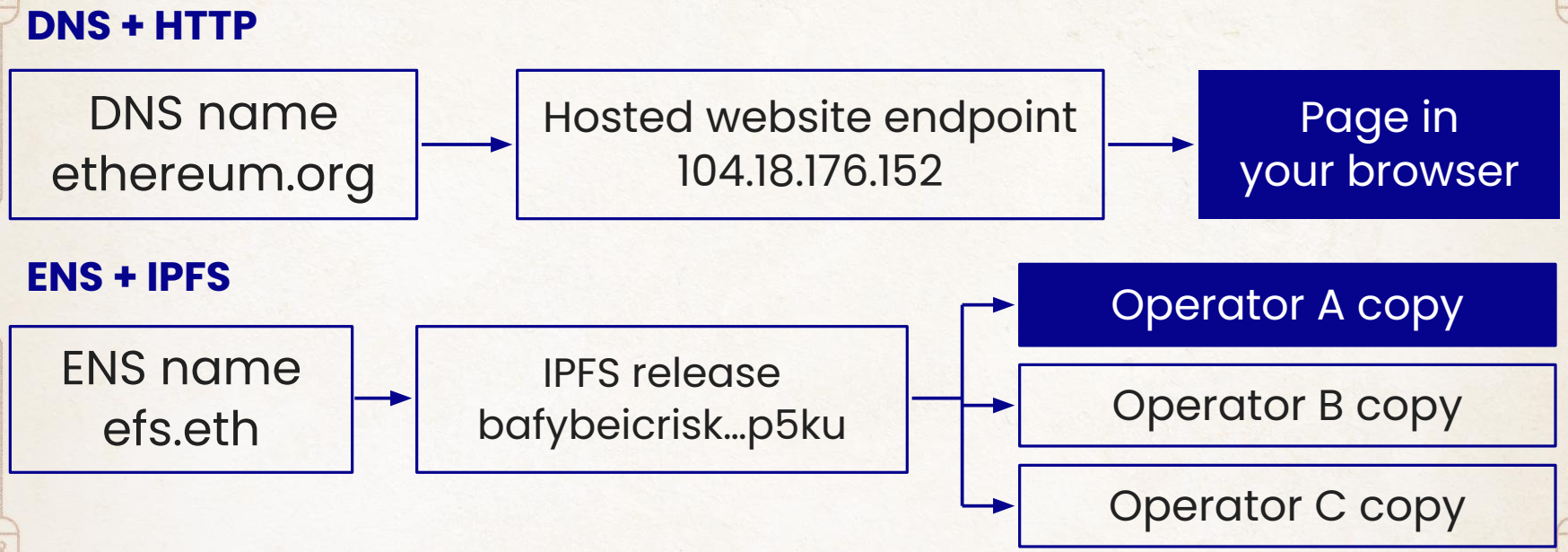
Find a saved release

Run it without your servers

Recover access without your help

Can people find and use your app?

More than one way into the app



IPFS gateways: efs.eth.limo (or .link) or your own IPFS node

A release others can run

What if the page opens but its scripts are gone?

Runnable release
HTML + JavaScript
Styles + images
Config + loaded code

↓

A user opens the app

Build materials
Source code
Locked dependencies
Build instructions

↓

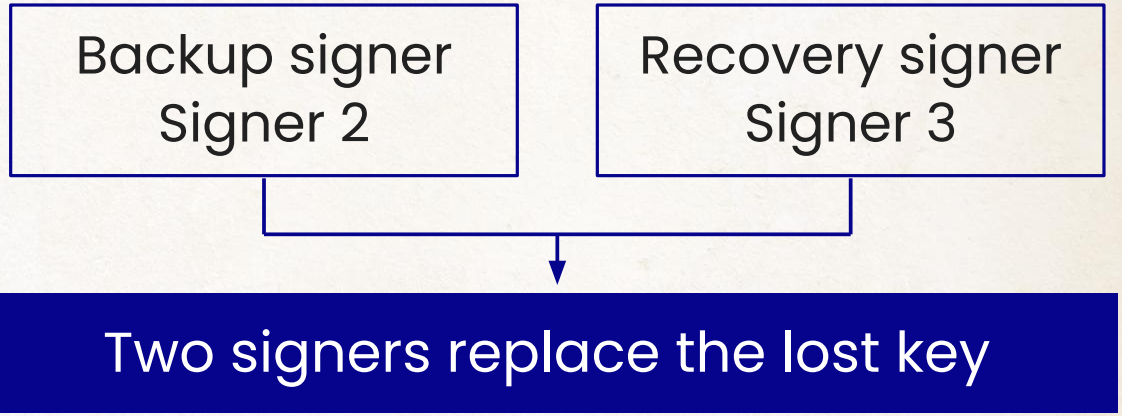
A developer rebuilds it

Check both paths on a fresh machine

Access people can recover

Safe example: two of three signers

Lost phone
Signer 1 offline



Same account
Same permissions

Practice lost-device recovery. Back up private-file keys too.



Data

Continuing the work

Is the data portable?

Keep the files and their meaning

Use them through another app

Let others run the supporting services

Data another app can understand

Example shared record	Shared meaning
id: note-17 author: 0xA... text: Meeting at 10 file: agenda.md (rev 2)	What each field means Who can change it How versions fit together How users decrypt it

EAS uses schemas. Existing databases can offer documented exports.

Same data, different apps

Notes app

Meeting at 10

note-17 by 0xA…

agenda.md (rev 2)

Community feed

note-17 by 0xA…

Meeting at 10

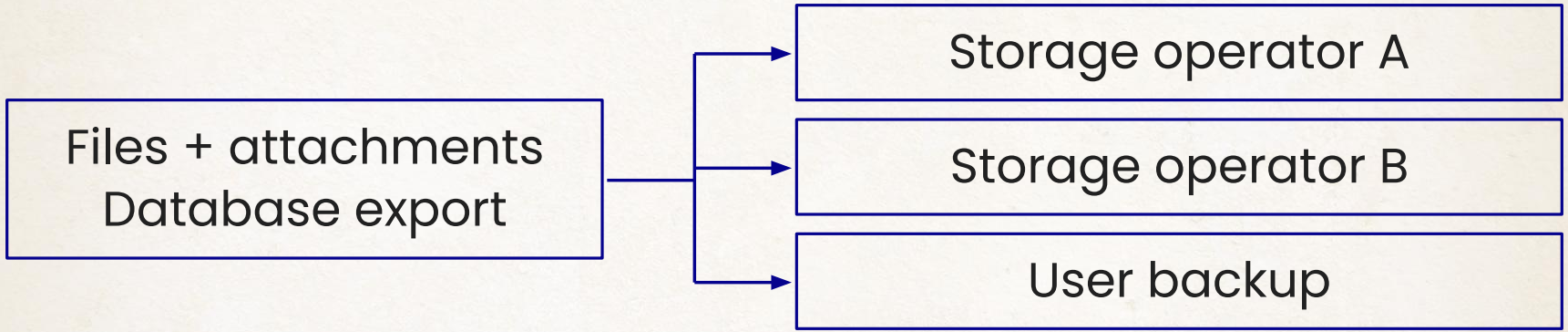
Open agenda.md (rev 2)

Same record ID, author and attachment

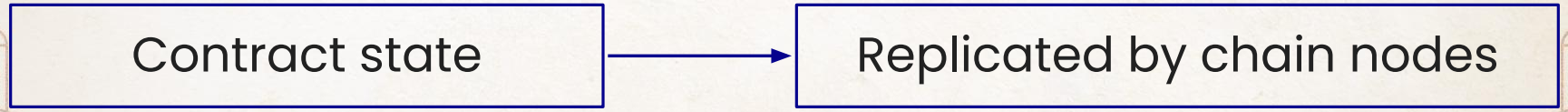
Illustrative interfaces using the same documented data

Who keeps a usable copy?

Files and databases

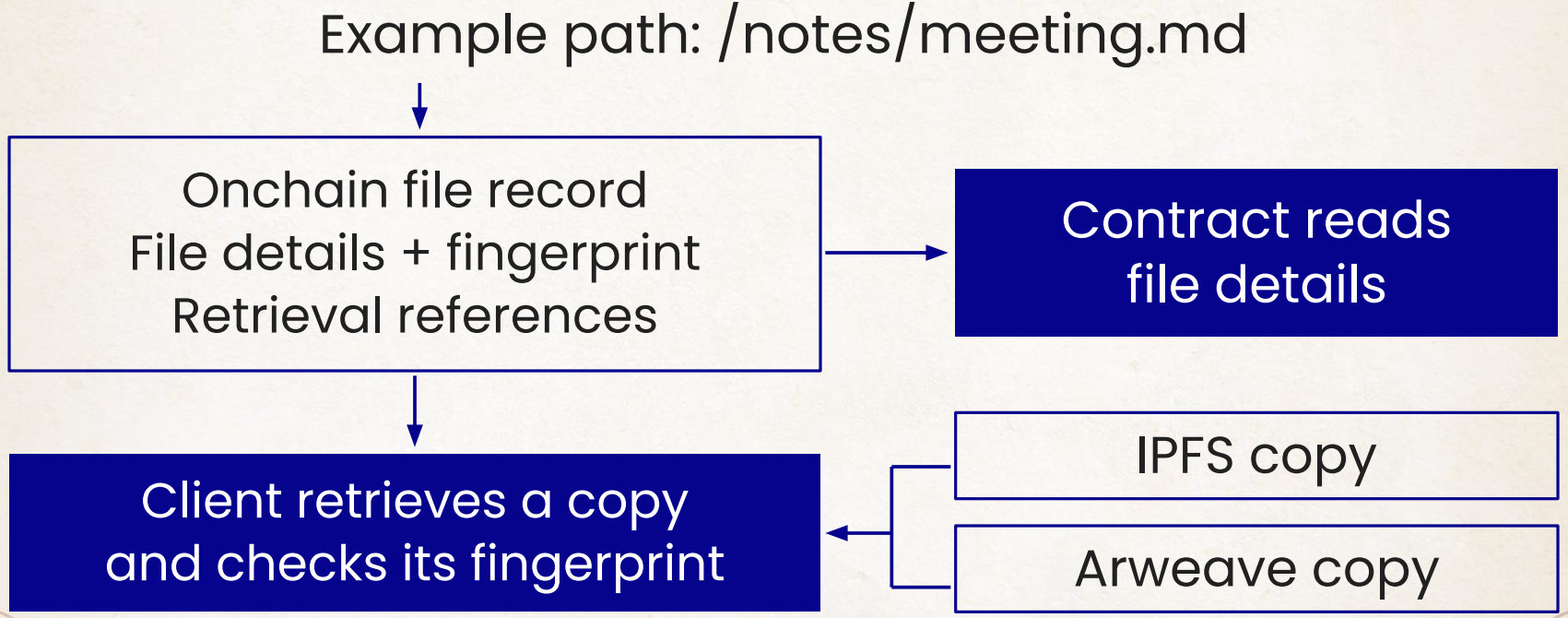


Onchain records



Check: restore useful work through an alternative

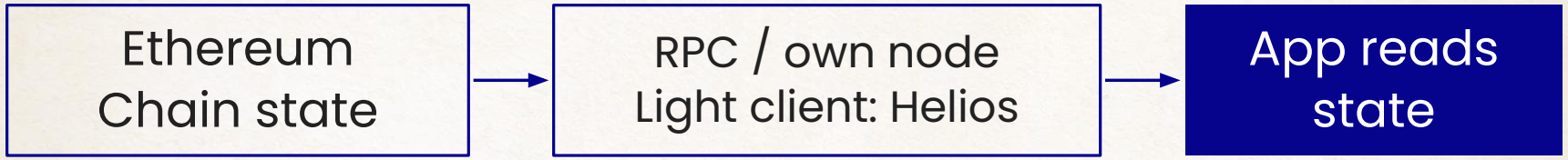
EFS: find a file and check a copy



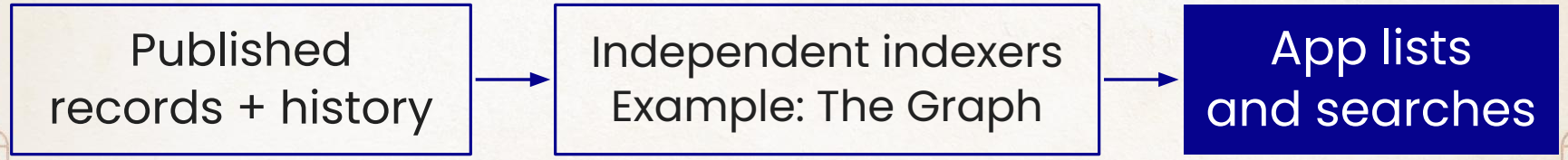
Same accepted file. A different provider or app can use it.

Reading and finding shared data

Read chain state



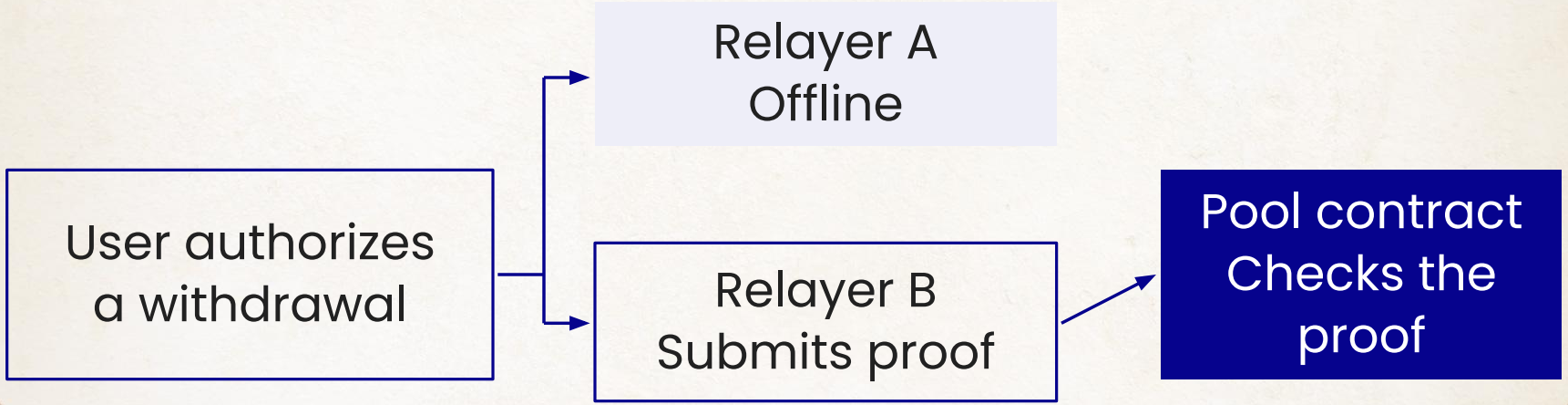
Find records in lists and searches



Check: change providers, rebuild a view and check its freshness

Who sends the next transaction?

Tornado Cash Classic: replaceable relayers



Check: could another operator complete your background job?



Rules

Keeping dependable rules

Stable core contracts

User-chosen app upgrades

Clear rules for shared governance

Who chooses the changes?

Immutable contracts

A stable core

Keep the contract code fixed

Let other apps build on it

Room to improve

Deploy a new version

Let users choose to migrate

Uniswap v2 core: fixed code, shared exchange logic

The network beneath the app

Ethereum and L1

Keep a working node route

Retain the history you need

Understand inclusion and finality

L2 and settlement

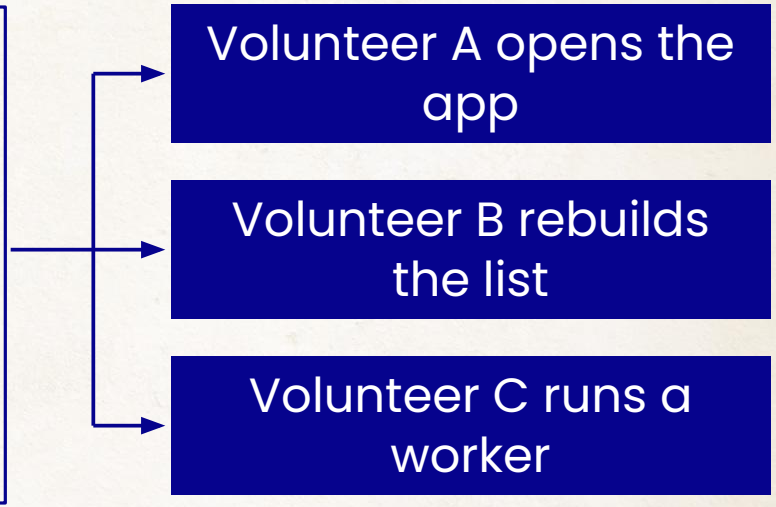
Know who orders and proves

Find the required chain data

Practice the recovery route

A community can carry it forward

Public handoff
Release + contract addresses
Retained data + index recipe
Worker code + running costs



Illustrative handoff: let people outside your team follow the instructions



Putting this into practice

A walk-away test for your app

Your walk-away checklist

A user can:

Run a saved app release

Recover access and keys

Restore usable data

Choose another client

Another operator can:

Switch RPCs; rebuild views

Run workers; keep copies

Inspect admin powers

Use network recovery

Try it without your team. Improve one dependency.

A walk-away test in practice

Find an existing note and open its attachment

1. Before	2. Index offline	3. After repair
Meeting at 10 agenda.md	Cannot load list Index unavailable	Meeting at 10 agenda.md

Repair: publish the index recipe and allow endpoint switching

Rebuild from retained records. Retry and open the attachment.

Illustrative test, not a recorded outage

Our work can outlive us


Thank you!
Let’s build reliable and long-lasting apps together
James Carnley
Ethereum File System
efs.eth.limo
JamesCarnley@gmail.com
@JamesCarnley